The Storage Layer Behind Kubernetes Database Operators That Enables Real-World DBaaS



Percona Live 2026 Amsterdam

George Kechagias, Senior Software Engineer, Percona

The next 50 minutes

Database storage on Kubernetes: the layer underneath your operator, and the one object it cannot rebuild for you.

The layer under the layer	what an operator builds, and the one object it cannot rebuild
What you configure	four lines of YAML, and the capabilities you buy with them
What the operator does with it	one claim becomes N volumes, and how it grows them safely
What Kubernetes does underneath	the objects, the CSI calls, and where a volume gets stuck
When the storage is slow	seven layers, four numbers, and what to check on Monday

PERCONA LIVE

© 2026 Percona

The layer under the layer 🧱

PERCONA LIVE
© 2026 Percona

Everything the operator layer gives you

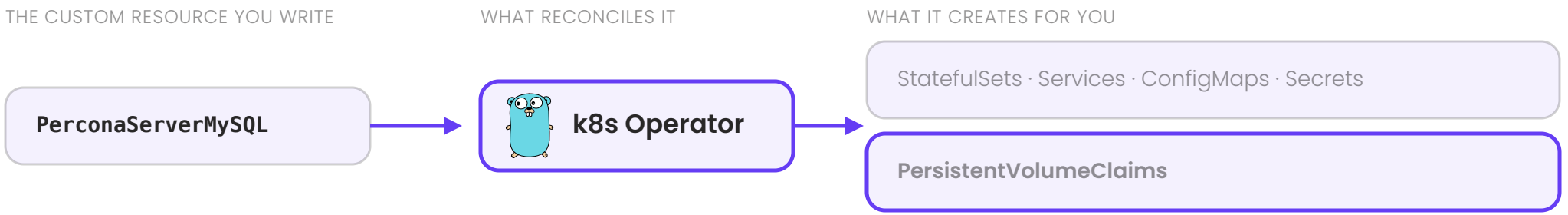
- **Control loops**: failover, backup and scaling run as code, continuously
- **A CR per cluster**: one declarative, versioned API object, validated by the API server
- **The platform's own objects**: RBAC, Secrets, StorageClasses, and the wider ecosystem
- **Portable by construction**: GKE, EKS, AKS, OpenShift, bare metal — same CR, local classes
- **A fleet you can review**: every database in the estate is a file in a git repository

PERCONA LIVE

© 2026 Percona

A Kubernetes Operator is your runbook, on a loop

Every example is the Percona Server for MySQL operator (PS). The shapes are the same for PXC, PostgreSQL and MongoDB.




```yaml
apiVersion: ps.percona.com/v1
kind: PerconaServerMySQL
spec:
  mysql:
    size: 3
    volumeSpec:
      persistentVolumeClaim:
        resources: {requests: {storage: 100Gi}}
```

PERCONA LIVE

© 2026 Percona

So what is actually on that storage?

DURABLE · ON THE PV

`/var/lib/mysql`

table data

redo · undo · doublewrite

binlogs

operator state files · mysql.sock

EPHEMERAL · GONE WITH THE POD

`/opt/percona`

the operator's binaries, reinstalled every start

`/.mysqlsh · /var/log/xtrabackup`

shell config · backup logs

SOMEWHERE ELSE ENTIRELY

config → ConfigMap + Secret

credentials → Secret

backups → object storage

none of it on your volume

PERCONA LIVE

© 2026 Percona

Six ways your storage fails you

it fills up

the engine stops writing,
then stops starting

it gets slow

and nothing inside the
database explains why

it won't attach

the pod sits in
`ContainerCreating`

it won't grow

the class said no,
months ago

it's in the wrong zone

so the replica waits.
it does not move.

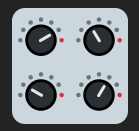
it gets deleted

by a scale-down, or a
reclaim policy nobody read

PERCONA LIVE

© 2026 Percona

What you configure



© 2026 Percona

The whole storage provisioning API is a couple of lines

```
volumeSpec:
  persistentVolumeClaim:
    # storageClassName: db-storage
    # volumeAttributesClassName: high-perf
    resources: { requests: { storage: 100Gi } }
```

- It's a plain `PersistentVolumeClaimSpec`, nothing operator-shaped about it, to build the PVCs.

THE OPERATOR ACCEPTS THREE · IT PICKS THE FIRST ONE YOU SET, IN THIS ORDER

`persistentVolumeClaim`

a real disk, managed by Kubernetes

survives reschedule to another node

`hostPath`

a directory on that node, as-is

survives the pod, never leaves the node

`emptyDir`

scratch space, created empty

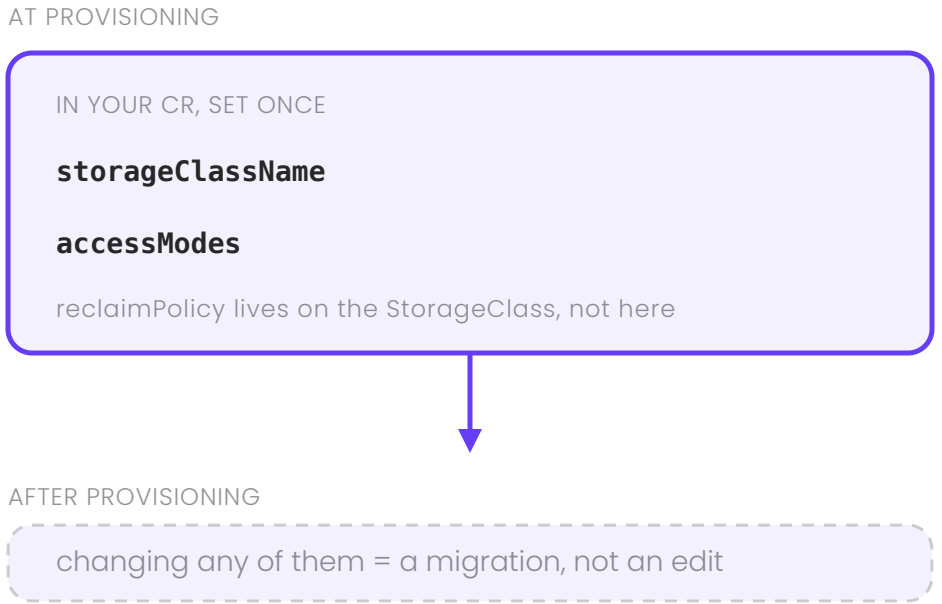
deleted with the pod. all of it.

PERCONA LIVE

© 2026 Percona

Some PVC fields cannot change that easily

- `storageClassName`: fixed for the life of the claim
- `accessModes`: a promise the driver keeps or quietly ignores
- `kubectl delete pvc` takes the disk with it, the class's `reclaimPolicy` decides



PERCONA LIVE

© 2026 Percona

Four ways storage gets to your pod

WHAT THE NODE IS HANDED	WHAT IT IS	WHO OWNS THE FILESYSTEM
a disk on the node	`local NVMe · LocalPV · hostPath`	your kernel. dies with the node.
a block device, remote	`EBS · PD · Ceph RBD · iSCSI`	your kernel formats it. RWO.
a filesystem, remote	`EFS · Filestore · NFS · CephFS`	their server owns it. RWX.
an HTTP API, never mounted	`S3 · GCS · Azure Blob`	nobody. it is not a filesystem.

PERCONA LIVE

© 2026 Percona

The Kubernetes object you probably didn't write

```
kubectl get storageclass              # what your cluster already has
kubectl apply -f db-storage.yaml      # and nothing stops you adding your own
```

```
metadata: { name: db-storage }          # kind: StorageClass · storage.k8s.io/v1
provisioner: ebs.csi.aws.com            # which driver, and therefore which world
parameters: { type: gp3, iops: "6000", throughput: "250" }
allowVolumeExpansion: true              # or your disk never grows
volumeBindingMode: WaitForFirstConsumer # or it is created in the wrong zone
reclaimPolicy: Delete                   # or kubectl delete pvc takes the disk
```

- Managed clusters ship some already: e.g. `gp2` (EKS), `standard-rwo` (GKE), `default` (AKS)
- `volumeSpec.persistentVolumeClaim.storageClassName: db-storage`

PERCONA LIVE

© 2026 Percona

You pick a storage name. You inherit a capability set.

THE NAME YOU SET IN THE CR

db-storage

→ names a CSI driver

WHICH DECIDES WHAT IS EVEN POSSIBLE

- allowVolumeExpansion
- snapshots · clones
- volumeBindingMode
- parameters: iops, throughput

INHERITED BY

every single claim made against it, with no way to opt one of them out

PERCONA LIVE

© 2026 Percona

The provisioner is a driver, and the driver picks the features

- `provisioner:` names the CSI driver that creates the disk — a program, not a K8s feature
- Kubernetes never calls the cloud provider: it calls the driver, the driver calls its API
- Not installed? every claim against that class sits Pending, see: `kubectl get csidrivers`

PROVISIONERS YOU WILL ACTUALLY MEET

`ebs.csi.aws.com`	`pd.csi.storage.gke.io`	`rook-ceph.rbd.csi.ceph.com`
`nfs.csi.k8s.io`	`disk.csi.azure.com`	`kubernetes.io/no-provisioner`

the class name is chosen by a human. this line is a fact about what your cluster can do.

PERCONA LIVE

© 2026 Percona

The newest storage object: change IOPS, not size

```
kubectl get volumeattributesclass     # GA in 1.34, locked in 1.36
```

```
kind: VolumeAttributesClass
metadata: { name: silver }              # another label you point a claim at
driverName: pd.csi.storage.gke.io       # the same driver that provisioned the disk
parameters: { iops: "3000", throughput: "150" }        # driver-specific names
```

- Point a live claim at it and the disk is modified in place — no resize, no new volume
- The parameters are immutable: repoint the claim at another class, don't edit this one

PERCONA LIVE

© 2026 Percona

But sometimes you will buy IOPS by the gigabyte



GROWING NEEDS allowVolumeExpansion · SHRINKING IS A MIGRATION, ALWAYS

GKE EXAMPLE · A CLUSTER BUILT ON N1 NODES

`standard-rwo · premium-rwo → pd-balanced / pd-ssd`

`nodes: n1-standard-8 (N1, 1st-gen)`

IOPS and throughput derive from the size

want more IOPS? buy more GB.

GKE EXAMPLE · WHAT YOU WOULD NEED INSTEAD

`hyperdisk-balanced`

`nodes: C3 · C4 · N4 · M3 (N1/N2/E2 gated)`

IOPS and throughput are their own fields

and a VolumeAttributesClass can change them

PERCONA LIVE

© 2026 Percona



What the operator does with it 🤖

PERCONA LIVE
© 2026 Percona

You apply the CR. This is what you watch.

```
→  percona-server-mysql-operator git:(main) k get pods -w
NAME                                               READY   STATUS    RESTARTS   AGE
percona-server-mysql-operator-7dbdfc6dd4-n6q2k     0/1     Running   0          10s
ps-cluster1-mysql-0                                0/2     Pending   0          0s
ps-cluster1-mysql-0                                0/2     Pending   0          0s
ps-cluster1-mysql-0                                0/2     Pending   0          2s
ps-cluster1-mysql-0                                0/2     Init:0/1   0          2s
percona-server-mysql-operator-7dbdfc6dd4-n6q2k     1/1     Running    0          18s
ps-cluster1-mysql-0                                0/2     Init:0/1   0          12s
ps-cluster1-mysql-0                                0/2     PodInitializing   0          16s
ps-cluster1-mysql-0                                1/2     Running           0          61s
ps-cluster1-mysql-0                                1/2     Running           0          84s
ps-cluster1-mysql-0                                2/2     Running           0          84s
ps-cluster1-mysql-1                                0/2     Pending           0          0s
ps-cluster1-mysql-1                                0/2     Pending           0          0s
ps-cluster1-mysql-1                                0/2     Pending           0          2s
ps-cluster1-mysql-1                                0/2     Init:0/1          0          3s
```

`kubectl get pods -w` on a fresh three-node cluster. Ordinal 0 has to be Running before ordinal 1 is even created.

PERCONA LIVE
© 2026 Percona

From your CR to a bound claim

ONE PASS OF THE RECONCILE LOOP

you apply the CR

`mysql.volumeSpec`

→

it builds a StatefulSet

`volumeClaimTemplates[datadir]`

it writes a template, not a volume

→

the StatefulSet makes the PVCs

`datadir-cluster1-mysql-0…2`

one per ordinal, and stable for it

the StatefulSet, the Services, the Secrets are owned by your CR · the claims are not

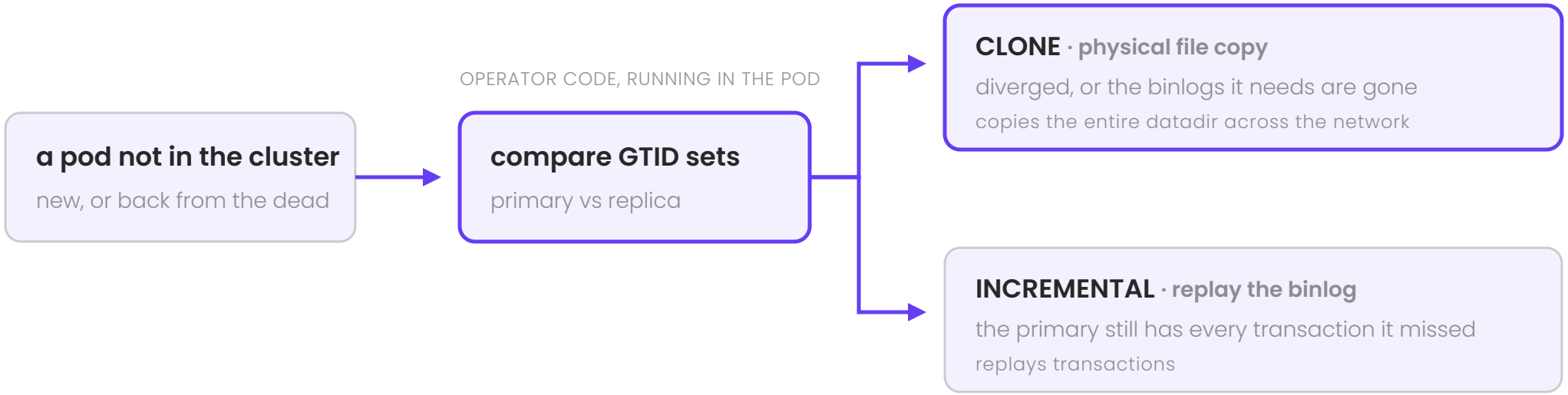
delete the cluster and they stay behind — removing them is opt-in, one finalizer you have to enable

from here the operator only comes back for one thing: to make them bigger

PERCONA LIVE

© 2026 Percona

The operator orchestrates what data a new replica gets



PERCONA LIVE

© 2026 Percona

The operator keeps its own state on your volume

WRITTEN INTO /var/lib/mysql, NEXT TO YOUR DATA

`bootstrap.lock`
initialising right now. wait.

`clone.lock`
this data came from a clone

`cr-version`
which CR version last started it

AND THE ONES THAT CHANGE HOW IT COMES UP

`full-cluster-crash`
starts, but read-only

`sleep-forever`
mysqld never starts at all

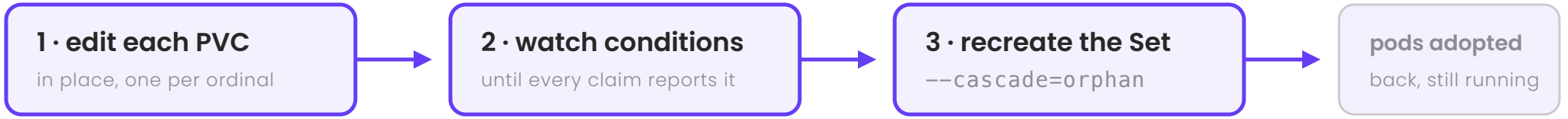
`clusterset-recovery`
wipes the other flags

the CR cannot hold this. it has to survive the pod, so it lives on the only thing that does.

a perfectly healthy volume can still refuse to come up

PERCONA LIVE
© 2026 Percona

Growing the volume, without the downtime




```
storageScaling:
  enableVolumeScaling: true       # nothing resizes without this
  autoscaling:
    enabled: true
    triggerThresholdPercent: 80   # default; clamped to 50–95
    growthStep: 2Gi               # default
    maxSize: 100Gi                # optional ceiling
```

PERCONA LIVE

© 2026 Percona



What Kubernetes does underneath ⚙

PERCONA LIVE
© 2026 Percona

Why your storage driver is not part of Kubernetes

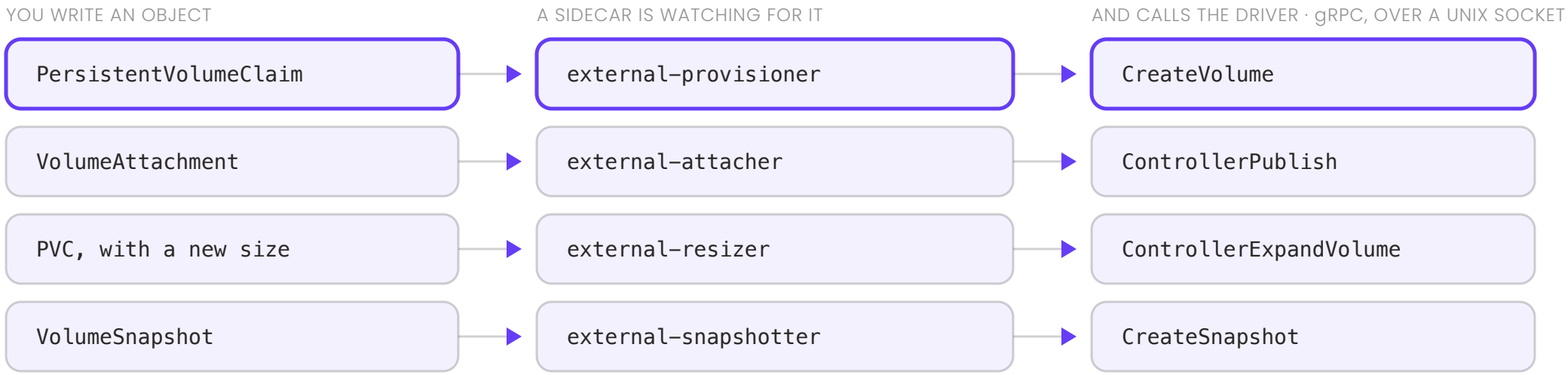
- **Container Storage Interface**: one gRPC API for block and file storage, any orchestrator
- Before it, drivers lived **in-tree**: fixes waited for a Kubernetes release · GA in 1.13
- Vendors write the drivers (AWS, Google, Ceph) and ship them as pods in your cluster
- Most CSI calls are optional: your feature set is whatever that driver advertises
- Kubernetes' half is sidecar containers: `external-provisioner`, `external-attacher`, `external-resizer`, making the CSI calls that the driver answers

PERCONA LIVE

© 2026 Percona

Kubernetes <> CSI <> storage

Those optional calls, and who makes them.



PERCONA LIVE

© 2026 Percona

From claim to mount, and where it hangs

PVC stuck in Pending

provision
`CreateVolume`

bind
`no CSI here`

attach
`ControllerPublish`

WHY IT STOPS IN THIS HALF

the claim never became a volume: no class, no capacity, a quota said no, or nothing has been scheduled yet

`kubectl describe pvc`
its events name the class, the quota, the provisioner

`kubectl get storageclass`
does the class it asks for actually exist?

Pod stuck in ContainerCreating

stage
`NodeStage`

publish
`NodePublish`

pod starts
`it has a disk`

WHY IT STOPS IN THIS HALF

the volume exists, but this node cannot take it: attach limit, a failed mount, or fsGroup still chowning files

`kubectl describe pod`
its events name the node, the attach, the mount

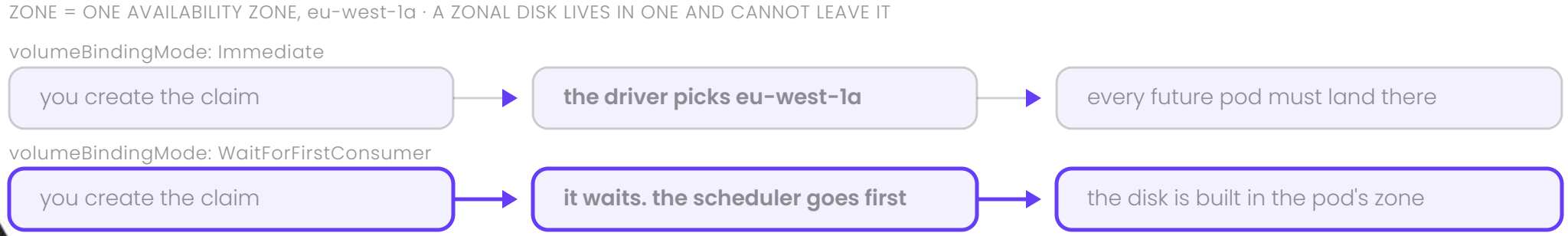
`kubectl get volumeattachment`
did the disk ever reach this node at all?

stage runs once per node, publish once per pod · when events run dry, the sidecar logs carry the driver's own error text

PERCONA LIVE
© 2026 Percona

Your volume pins your pod

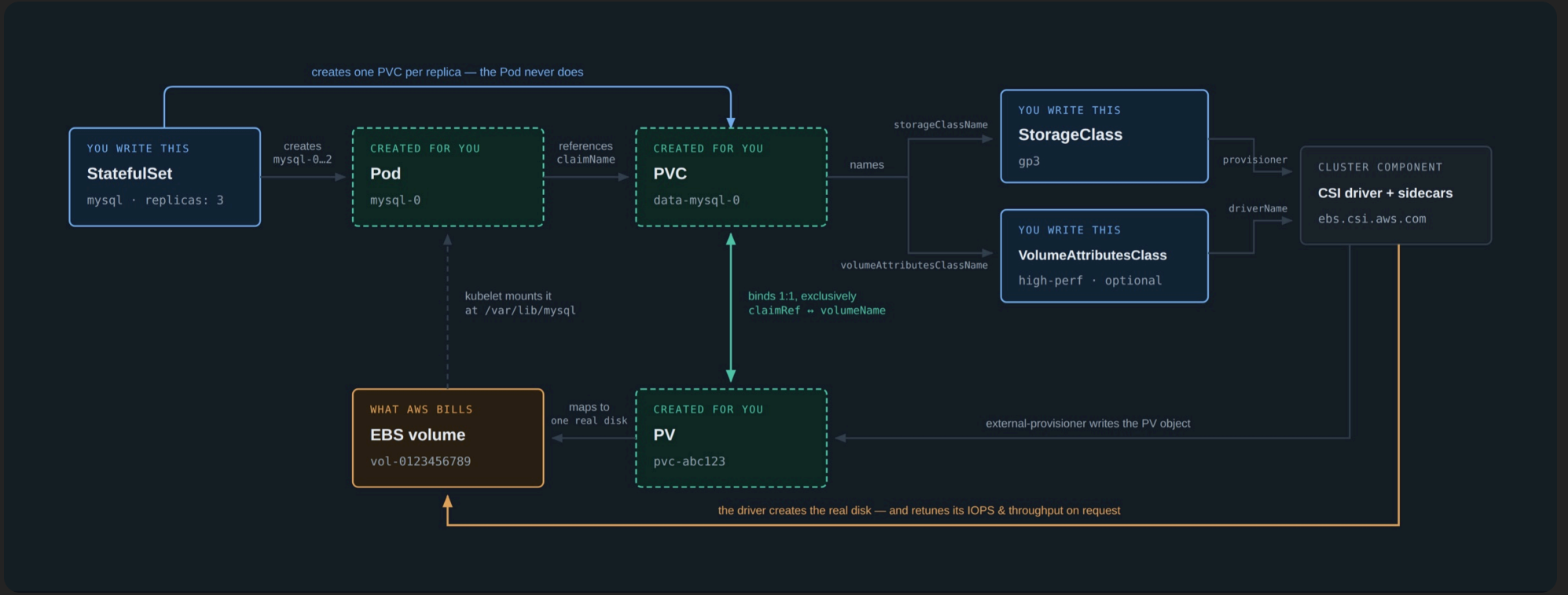
- One line on the **StorageClass** decides this: `volumeBindingMode`
- `Immediate` builds it now; `WaitForFirstConsumer` waits for the scheduler.
- The PV keeps that zone forever; when it fills or fails: `volume node affinity conflict`



PERCONA LIVE

© 2026 Percona

Putting everything on the same picture



AWS EBS is just the worked example. The same chain holds on GCP, Azure and other providers. Only the provisioner and the disk change.

PERCONA LIVE
© 2026 Percona

Storage on Kubernetes FAQ: what actually happens?

"What if the CSI driver pods go down?"

running pods keep running, untouched
new pods cannot start: no attach, no mount
and none can leave: unmount needs it too

"Is CSI making my I/O slow?"

almost never: it is not in the data path
it only provisions, attaches and mounts
look at the backend, the tier, the network

"What if I force-delete the pod?"

the volume can stay attached to a dead node
and the replacement pod waits on it
`kubectl get volumeattachment`

"Can I delete a PVC that is in use?"

you can ask. it will not happen yet.
it sits in Terminating until no pod uses it
`kubernetes.io/pvc-protection`

PERCONA LIVE

© 2026 Percona

“The database is slow” 💀

PERCONA LIVE

© 2026 Percona

They own the storage stack. We don't.

- Cloud DBaaS providers can reason about storage because they **own the whole storage stack**
- Kubernetes operators don't: not the disk, not the driver, not the node
- So how close can we get with the signals and control surfaces Kubernetes gives us?

PERCONA LIVE
© 2026 Percona

No single layer sees the whole I/O path

EVERY ONE OF THESE CAN QUEUE

- engine buffer
- fsync and the page cache
- Linux page cache / filesystem
- block layer and cgroup throttling
- CSI, and the network under it
- cloud block-storage service
- provider-side queues / limits / throttling



the engine reports the total only

PERCONA LIVE
© 2026 Percona

Ask the engine, ask the device

ASK THE ENGINE

`performance_schema.file_summary_by_event_name`

dblwr write: avg 18.5 ms · max 408 ms

log fsync: avg 4.5 ms · max 273 ms

what the database experienced

ASK THE DEVICE

`grep sdb /proc/diskstats · inside the pod`

11,774 writes / 70,861 ms = 6.0 ms

what the kernel measured

the engine waits longer than the device reports

that gap is queueing and fsync, not the disk being slow. one number alone would have blamed the wrong layer.

PERCONA LIVE
© 2026 Percona

Ask Kubernetes, ask the cloud

ASK KUBERNETES

`nodes/<node>/proxy/metrics`

kubelet_volume_stats_*: 207 MiB of 1.9 GiB

cAdvisor: per-pod blkio, per device

capacity and attribution. no latency.

ASK THE CLOUD

`gcloud compute disks describe`

pd-balanced, 2 GB, no provisioned IOPS

performance_status: Healthy · Degraded

latency and throttling live here, not in the cluster

on GKE the CSI controller runs in the managed control plane

so there are no provisioner or attacher metrics to scrape, and no node-exporter means no node_disk_* either

PERCONA LIVE
© 2026 Percona

The one knob Kubernetes gives you, refused 💥

```
kind: VolumeAttributesClass          # storage.k8s.io/v1, GA
driverName: pd.csi.storage.gke.io
parameters: { iops: "3000", throughput: "150" }
```

- Patch it onto a bound claim and the driver answers, in its own words:

```
VolumeModifyFailed  rpc error: code = InvalidArgument desc =
  Failed to modify volume: modifications not supported for disk type pd-balanced
```


PERCONA LIVE
© 2026 Percona

The metrics that can mislead you

- `iostat %util` at 100% means the device was **busy**, not that it was saturated
- An average stays calm through the tail latency your users actually felt
- The node looks healthy while one cgroup is throttled to a crawl
- Free space looks fine while size-derived performance is the real limit

PERCONA LIVE
© 2026 Percona

From signals to a storage decision



PERCONA LIVE
© 2026 Percona

What the operator can actually do

- Grow the volume before it fills, and record `currentSize`, `resizeCount`, `lastError` in status
- Surface storage failures as events on the CR, not buried in kubelet logs
- Ensure per-pod storage signals are collected before the incident
- Run backups from a replica, so durability stops competing with production reads
- Name the driver's refusal, `StorageClassNotSupportResize`, instead of retrying blindly

PERCONA LIVE
© 2026 Percona

What no operator can do for you

exceed what the backend offers

a VolumeAttributesClass can retune it, if the disk type allows

change a bound PVC's StorageClass

that is a migration, and always was

make cross-AZ latency smaller

than the distance

fix a snapshot after you took it

crash-consistent cannot become application-consistent later

PERCONA LIVE

© 2026 Percona

Ideas to try

- `kubectl get sc -o wide`: provisioner, binding mode, expansion, in one line
  - three of the six failure modes were decided here, long before the incident
- `kubectl -n kube-system get ds,deploy | grep csi`: find your driver's pods
  - when a volume hangs, this is how you know whose logs to read and whose ticket to open
- **Ask MySQL how long a write took, then ask the disk**
  - MySQL waited 18 ms, the disk took 6. The other 12 never touched the disk
- **Point a `VolumeAttributesClass` at a claim you don't care about**
  - either it retunes the disk, or you learn today that your disk type has no knob

PERCONA LIVE

© 2026 Percona

Q&A. And what ships next.

ALL OF THIS IS OPEN SOURCE, INCLUDING THE BITS I COMPLAINED ABOUT

github.com/percona

MYSQL

`percona-server-mysql-operator`

`percona-xtradb-cluster-operator`

POSTGRESQL

`percona-postgresql-operator`

MONGODB

`percona-server-mongodb-operator`

THE OPERATORS' PUBLIC ROADMAP

`github.com/orgs/percona/projects/10`

PERCONA LIVE

© 2026 Percona